

SPRINT 4.1 COMPLETION REPORT

SHADOW OS v2 — Production PostgreSQL Persistence

' COMPLETE — 4/4 TESTS PASS

Date: 2026-07-11 · Sprint 4.1 · FOREX ENGINE PRO

Executive Summary

Sprint 4.1 closes the single most dangerous gap in the production deployment:

****Railway had no PostgreSQL service and no DATABASE_URL, so every deploy wiped all accumulated trading knowledge.**** This sprint makes PostgreSQL the primary production database, guarantees the full SHADOW OS v2 schema is created automatically and idempotently on every boot, and proves — via test — that re-running startup never erases data.

No filesystem or SQLite-volume workaround was used. SQLite remains a local-development-only fallback. After the operator attaches a Railway PostgreSQL service and redeploys, all future trading history, memory, snapshots and recovery data persist across every deployment.

****Verdict: ' COMPLETE — architect review PASS on all 9 requirements, 4/4 verification tests passing.****

Problem Statement

The domain-manager tier (RuntimeDomainManager, TradeIntentManager, MemoryManager) and LiveMemoryIntegration are **PostgreSQL-only** and assume

their tables already exist. Those tables were only ever created by running the migration SQL files manually through `psql -f (telemetry/migrations/run.js)`.

Consequences on a fresh Railway PostgreSQL service:

- The SHADOW OS v2 tables would **not** exist.
- `LiveMemoryIntegration.init()` would throw on the first manager query and the entire memory/snapshot/recovery layer would silently degrade to no-op.
- With no `DATABASE_URL` at all, the legacy telemetry (events, `shadowm_trades`) would fall back to ephemeral SQLite and be lost on redeploy.

Root cause of "data lost every deploy": ****DATABASE_URL was never set in production, and even once set, nothing created the schema automatically.****

What Changed

1. Startup-safe auto-migration runner — `telemetry/migrations/autoMigrate.js`

A new module exporting `ensureSchema(pool)`:

- Runs the four migration files (001–004) in order, applying only those not yet recorded in a new `schema_migrations` tracking table.
- Executes **the entire file** via `pool.query(fileContents).node-postgres'`

simple query protocol (a query string with no bind parameters) lets the

server

parse statement boundaries — including DO \$\$... \$\$; blocks — and

runs each file as **one implicit transaction (all-or-nothing per file)**

. No

psql binary is required (it is not guaranteed to exist in a Railway Nixpacks

container), and no fragile JS "split on ;" splitter is used.

- Serializes concurrent migrators with a **session advisory lock** (21320, 40911) — deliberately distinct from LiveMemoryIntegration's recovery lock (21320, 20307) — so overlapping old/new processes during a Railway redeploy cannot race.
- Is fully **idempotent** and **data-safe**: every migration uses IF NOT EXISTS / ADD COLUMN IF NOT EXISTS / ON CONFLICT DO NOTHING, and there is no DROP TABLE / TRUNCATE / DELETE anywhere in the migration set.

2. Wired into startup — telemetry/managers/LiveMemoryIntegration.js

ensureSchema() runs inside init() **after**

the pool is created and

before the managers are constructed, gated on this._ownPool:

- On real server startup, server.js constructs the integration with no injected pool, so _ownPool = true and the schema is ensured on every boot.
- Tests inject _pool (_ownPool = false) and skip auto-migration.
- A failure propagates to the existing init() catch, which degrades the memory layer to no-op — **a migration failure can never block trading** (sacred constraint preserved).

The last migration summary is stored and surfaced through getStatus().

3. Persistence health endpoint — telemetry/server.js

GET /api/healthz/persistence now reports:

- dbBackend: "postgresql" and persistence: true when on PostgreSQL,
- a storage descriptor with a plain-language persistence note,
- the full memoryIntegration status (including the schema applied/skipped summary), so the endpoint confirms PostgreSQL is active end-to-end.

Requirements Compliance

#	Requirement	Status
1	Railway PostgreSQL as primary production DB	' (db-adapter selects PG on DATABASE_URL)

2	Auto-use DATABASE_URL when present	' (unchanged, verified USE_PG=true)
3	SQLite only as local dev fallback	' (fallback only when no PG URL)
4	Auto-create tables/migrations on startup	' (ensureSchema in init())
5	Never erase existing data on startup	' (idempotent DDL, no DROP/TRUNCATE/DELETE; test proves survival)
6	Reports dbBackend="postgresql" + persistence=true	' (health endpoint)
7	/api/healthz/persistence confirms PostgreSQL active	' (extended endpoint)
8	No filesystem / SQLite volume solution	' (none added)
9	Trading history/memory/snapshots/recovery persist across deploys	' (schema ensured; managers are PG-only)

Verification

Test suite: telemetry/tests/integration/autoMigrate.test.js (run against the development PostgreSQL).

```
' first run reports ok and leaves every expected table present
' second run is a no-op — applies nothing, skips all migrations
' re-running ensureSchema (simulated redeploy) never erases data
!9 tests 4 !9 pass 4 !9 fail 0
```

- All 15 expected tables (14 domain tables + schema_migrations) present after ensureSchema.
- Second run applies zero migrations, skips all — idempotency proven.
- A sentinel row written to events survives a simulated redeploy (re-run of ensureSchema); row count preserved — sacred constraint proven.

Backend selection confirmed in the dev environment:

USE_PG = true !' dbBackend = "postgresql" !' persistence = true.

Residual Notes (non-blocking)

Raised by the architect review and carried forward:

1. First true fresh-DB creation happens on the real Railway boot.

The dev

database already contained every table, so the tests exercised

re-application, not creation-from-empty. The same migration files bootstrap

fresh databases through run.js, so risk is low, but the operator should

confirm GET /api/healthz/persistence on the first Railway deploy.

2. Latent events schema-ownership drift.

telemetry/index.js's

_initSchema (run at require time) defines events.data TEXT NOT NULL, while

migration 001 defines events.data JSONB. Whichever `CREATE TABLE IF NOT

EXISTS runs first wins; on a fresh boot `_initSchema`` almost certainly wins, so production events.data will be TEXT. Harmless today — no code applies JSONB operators to events.data — but the two definitions should eventually be reconciled to a single owner.

3. With `SHADOW_OS_MEMORY=off`, v2 schema auto-creation does not run (it is coupled to the memory flag). This is consistent with the documented "off = zero behavior change" contract.

Operator Runbook — Enabling Production Persistence

1. In the Railway project, add a **PostgreSQL** service (plugin). Railway injects `DATABASE_URL` into the app service automatically.
2. Redeploy the app service. On boot the app will:
 - detect `DATABASE_URL != 'select PostgreSQL,`
 - run `ensureSchema != 'create all SHADOW OS v2 tables idempotently,`
 - initialize the memory/snapshot/recovery layer.
3. Confirm with `GET /api/healthz/persistence` — expect `dbBackend: "postgresql"`, `persistence: true`, and a `memoryIntegration` block with a schema summary.
4. From then on, all trading history, memory, snapshots and recovery data persist across every deployment.

Files Touched

File	Change
telemetry/migrations/autoMigrate.js	New — startup-safe idempotent schema runner
telemetry/managers/LiveMemoryIntegration.js	Auto-run <code>ensureSchema</code> in <code>init()</code> ; expose schema in <code>getStatus()</code>
telemetry/server.js	<code>/api/healthz/persistence</code> reports persistence + storage + integration status
telemetry/tests/integration/autoMigrate.test.js	New — idempotency + data-safety verification
docs/reports/SPRINT_4_1_REPORT.md (+ .pdf)	This report
CHANGELOG.md, replit.md, docs/architecture/MASTER_ARCHITECTURE.md	Documentation updates

Untouched (per constraints): `index.js`, `railway.json`.

FOREX ENGINE PRO · SHADOW OS v2 Migration · Sprint 4.1 · 2026-07-11